

Certifying a Typed Ledger: The Predicates, the Programmes, the Vintages, and the Reproduction Bundle

Amin Abaee

Independent Researcher

ORCID: 0000-0002-0019-1842

amin_abaee@ut.ac.ir

September 17, 2026

A methods companion to "Typed Flux Ledgers and Depletion Arithmetic: Conservation, Componentwise Diagnostics, and the Semantics of Depletion Horizons" (main text v39; supplementary v10).

Track. Methods and software. **Numbering convention.** This companion numbers its procedures **Protocol 1–8** in a single sequence and claims no theorem: every correctness statement it makes is either (a) quoted from the main text by label, (b) a statement about an algorithm’s behaviour, or (c) a value printed by the bundled code. Nothing here is proved afresh; nothing in the main text depends on anything here.

1. What this companion is for

The main text states eight certification obligations and three linear programmes as *results*: what they decide, what they cannot decide, and what a failure to satisfy one does and does not imply. It deliberately does not state them as *procedures*, because a journal article is not the place for input schemas, exit conditions, solver caveats and a run transcript.

This companion is that place. It answers four questions a reader who wants to *use* the apparatus has to answer without the help of a proof:

1. **What do I have to supply?** Section 3 gives the exact input schema: three tables, one vector, five declarations, nothing else. 2. **What does each check actually compute, and when does it stop?** Section 4 gives the eight predicates as decision procedures, with what each consumes and what its negative output means. 3. **What can I hand to a solver, and how do I read the answer?** Section 5 gives the three programmes with their input arrays, their duals where a dual exists, and the reading rule for infeasibility. 4. **How do I make the numbers checkable after the database moves?** Section 7 gives the vintage protocol: the three tests that establish which release a figure came from, and the two that a figure passes without it.

Scope discipline. Two claims are refused throughout. The companion does not present the predicates as sufficient for safety, because the main text proves they are not: Proposition 42 (no conservation law crosses a type class), Proposition 31 (aggregates do not transport component event times) and Proposition 30 (an aggregate can refute or alarm, never clear) are boundaries on what any checker can output, and this companion restates them as *properties of the tool* rather than pretending around them. Nor does it present any code as validating a published indicator: the bundled exhibits run on declared numbers, which is the whole point of Section 8.

Reading order. A reader who wants to run something goes to Sections 3, 5 and 8. A reader who wants to know what the tool cannot do goes to Section 4, then Section 9.

2. Status vocabulary, and why the tool has three negative outputs

A checker that prints "fail" for everything it cannot evaluate is worse than no checker, because it converts absence of a declaration into a refutation of the system. The main text therefore distinguishes three non-passing statuses (its Section 3.1, with the discharge details in the supplementary's S7 as corrected by S15 and S16), and the tool must implement all three:

Status	Meaning	Who is at fault
established	the obligation is discharged, with a named witness	—
not established	the obligation was not discharged on the declared data	the declaration is incomplete
not applicable refuted	the object cannot state this obligation at all the obligation's negation holds on the declared data	neither; the object is of a different kind the ledger, not the paperwork

The rule the implementation enforces is that **refuted** requires a witness of the negation, and that an undeclared obligation may never be reported as **refuted**. In practice this is the only place in this corner of the literature where a "0/8 passed" score is distinguishable from an "8/8 not stated" score, and the distinction is the difference between an audit and an accusation.

3. The input schema

Protocol 1 (Ledger input). A ledger instance is four objects and five declarations.

Objects:

#	name	shape	contents
1	compartment table	one row per compartment	identifier, type label, unit, boundary flag (interior or boundary), capacity bounds
2	flux table	one row per flux	source, sink, orientation, rate bound $\mathbf{v_bar}$, whether the flux is a transfer or a conversion
3	incidence operator S_T	derived, then checked	sparse $\mathbf{n_comp} \times \mathbf{n_flux}$, written from rows 1 and 2
4	boundary schedule B_T $\mathbf{u_partial}$	one column per boundary flux	the exogenous term of the typed balance $S_T \mathbf{v} + B_T \mathbf{u_partial} = \mathbf{b_dot}$

Declarations (each is a *named object*, not a flag): the conserved-covector list with its disturbance budget $\mathbf{eps_1}$; the margin map $G\mathbf{x} + \mathbf{a}$ with its service weights $\mathbf{c}$ and required rate $\mathbf{y_req}$; the barrier set and horizon; the declared disturbance or drift class; and, if affinity is to be checked, the species table and the set Ω . A missing declaration is not an error at this stage — it is the input to the **not established** status of Section 4.

Why a *type label* rather than a *unit string*: two compartments may share kg and still not be summable if they carry different moieties, and the main text's Definition 47 makes the type structure part of the ledger rather than part of a unit convention. The tool follows the text: **Typed** asks whether every compartment carries a declared $(Ty, \mathbf{ty}, Cv)$ triple and whether any column of the flux table is booked as both a transfer and a conversion; it does not ask whether the units look consistent.

Protocol 2 (what the input is *not*). The input schema has no field for "data source" and no field for a pull date. Those belong to the vintage record of Section 7, because they are checked against the release rather than against the ledger, and mixing the two lets a reviewer treat a stale citation as a modelling error.

4. The eight predicates as decision procedures

Protocol 3 (predicate battery). Each entry is a procedure over the objects of Protocol 1. "Cost" counts elementary operations on the declared tables; n_f is the number of fluxes, n_c the number of compartments.

Entry	Procedure	Cost	Stops with refuted when	Locus in the main text
Typed	scan the compartment table for a missing (Ty, ty, Cv); scan S_T for a column that is both transfer and conversion	$O(n_c + n_f)$	a row sums unlike types, or a conversion is booked as a sum	Def. 47, Prop. 42
Balanced	evaluate $S_T v + B_T u_{\text{partial}} - b\text{-dot}$ on the declared trajectory or on the published annual changes; residual must vanish to the stated tolerance	$O(\text{nnz})$	a named entry is missing, duplicated, or mis-routed	Lemma 3
Conserved	compute a basis of the left null space of S_T; for each vector, check the declared budget eps_1 is respected along the trajectory	rank factorisation, $O(n_c^2 n_f)$ worst case	a declared invariant changes by more than its budget	Prop. 4 with Thm. 24
Positive	check donor limitation: every primitive outflow must vanish when its donor is empty; equivalently test the Nagumo condition at the orthant boundary	$O(n_f)$ per flux, closed-form for affine ϕ_i	a flux leaves an empty compartment	Thms. 10–11
Admissible	for each conversion, test feasibility of the declared reaction and the sign constraints on energy and entropy terms; empty corridor $\Rightarrow$ name the conflicting pair	LP per conversion	a mass-balanced ledger that no chemistry realises	Defs. 1 and 42, Prop. 4
Safe	with the margin map and the horizon fixed, solve the multiplier programme of Section 5; report the bound or its absence	one LP	a barrier is crossed on a declared scenario within the horizon	Thm. 24; Defs. 4 and 11
Adequate service	evaluate the readout against the demand relation over the horizon	$O(n_f)$ per step	service is delivered by unpriced drawdown of support	Prop. 25

Closed	solve the closure programme of Section 5 for $\lambda \geq 1$ at the stated τ_{use} ; on failure, return the dual cut	one LP	demand is provably outside the closure cone	Defs. 21–22
--------	---	--------	---	-------------

Three implementation notes that the main text’s statements leave open, and that a user will hit.

Typed is the only predicate decidable from a declaration alone. Every other row needs a trajectory, a scenario set, or both. This is why the tool runs **Typed** first and reports the remaining seven as **not established** rather than **not applicable** when the trajectory is absent: the obligation is stated, it just cannot be discharged from the paperwork.

Positive and Safe interact in one direction only. **Safe** over a barrier set contained in the nonnegative orthant implies **Positive** over that horizon; the converse fails, and a checker that reports **Positive** as evidence of **Safe** has inverted an implication the main text proves one way. The tool’s status vector is therefore not a score: no monotone function of the eight entries reproduces the implications among them.

Admissible is optional by design, and the optionality is reported. The affinity clause needs a species table and a declared Ω ; where an instance supplies neither, **Admissible** is evaluated on the reaction feasibility and sign constraints only, and the tool prints which sub-clause was evaluated. Silently weakening a predicate is the failure mode this class of tool most often exhibits.

5. The three programmes: what to type into a solver

Protocol 4 (closure capacity, Definition 21). Over variables (v , λ):

```

maximise  $\lambda$ 
subject to  $S_T v + B_T u_{\text{partial}} = 0$ 
 $0 \leq v \leq v_{\text{bar}}$ 
 $\lambda D \leq P v$ 

```

Inputs: S_T , the capacity vector v_{bar} , the boundary schedule, the demanded use vector D , and the use-edge matrix P . Reading rule: feasibility with $\lambda \geq 1$ is the regime in which the cycle closes at the demanded rate; $\lambda < 1$ names the fraction at which it closes. The dual is the cut condition on return capacity (Gale, 1957; Ahuja et al., 1993): for every set of use edges, the return capacity into that set must cover the demand placed on it. The dual is the useful output when the primal fails, because it identifies the set of uses the returns cannot cover — a statement a reader can act on, rather than a status.

Protocol 5 (critical-margin multiplier, Theorem 24). Over variable λ :

```

find  $\lambda$ 
subject to  $\lambda^T G S_T + c^T \leq 0$ 
 $0 \leq \lambda \leq \lambda_{\text{bar}}$ 

```

The upper bound λ_{bar} is not a numerical convenience: it is what makes the budget finite, and it must be declared. If the system is feasible, the article’s budget inequality $T \leq V(x_0)/(y_{\text{req}} - \beta)$ is available with $\beta = \max \lambda^T G b$ over the declared box. **Infeasibility carries no conclusion:** it shows that this multiplier family does not close, and the tool must print that sentence rather than “no safety margin”, because the second is a claim about the ledger and the first is a claim about the search. Inputs: the affine margin map $Gx + a$, the weights c , the flux bounds, y_{req} , and the units assigned to each component of λ — an unlabelled multiplier is uninterpretable, since the reading of the bound depends on the unit of λ .

Protocol 6 (worst concealed deficit, Proposition 30). For each component j :

```

minimise b_j
subject to w^T b = z
l <= b <= u

```

and $\text{delta_j}(z) = -b_j^{\min}$. Inputs: the published aggregate z , the weights w , the component bounds l , u . The bounds are what a publisher must supply; their absence is why no numeric value for a public indicator is printed anywhere in the main text, and why this programme is the one a reviewer can actually run. Reading rule: **infeasible** means z lies outside the image of the declared box, which is a finding *about the disclosure*. Two quantities that look redundant are not: the compensation premium measures the trades that produced the aggregate, delta_j measures what the aggregate can hide. A single line of the exhibit in Section 5.4 shows both.

Solver notes. Any simplex or interior-point routine suffices; the bundled code uses `scipy.optimize.linprog(method="highs")`. It carries a fallback that enumerates vertices of the box intersected with the hyperplane, which is exact and slow: it is offered for environments without a solver and is valid only for small component counts ($2^{(n-1)}$ enumeration), and the code prints which route it used on every line. A reported optimum without a printed solver route is a reproducibility defect, so the print is part of the contract.

5.4 The exhibit, as printed

Two components, $w = (0.5, 0.5)$, declared bounds $l = (-4, -4)$, $u = (10, 10)$, from `code/certification_lp.py` (the run record is in Section 8):

```

published aggregate z = 0.0 : displayed split [0.0, 0.0] -> premium Pi = 0.00 ;
concealed deficits delta*_1 = 4.00 , delta*_2 = 4.00 (scipy.linprog(highs))
published aggregate z = 1.0 : displayed split [2.0, 0.0] -> premium Pi = 1.00 ;
concealed deficits delta*_1 = 4.00 , delta*_2 = 4.00 (scipy.linprog(highs))
published aggregate z = 3.0 : displayed split [6.0, 0.0] -> premium Pi = 3.00 ;
concealed deficits delta*_1 = 4.00 , delta*_2 = 4.00 (scipy.linprog(highs))

```

The third line is the one to read. A published value of 3 is compatible with a deficit of 4 in *either* component while the premium reports 3: the aggregate's optimism and the aggregate's concealment differ by a factor and are computed from different data, so no version of "report the premium" substitutes for "report delta_j ". The same script also prints the event-time witness of Proposition 31 — one aggregate trajectory $Z(t) = 100 e^{-t}$ from starts (2, 98) and (50, 50), with component barrier crossings at 0.6931 and 4.5850 in the first case and at 3.9120 in the second. Identical aggregates, different event times, no approximation anywhere in the calculation.

6. The output contract

Protocol 7 (report format). One JSON object per ledger instance, with the following keys and no others. The object below is a format example, not a run record; the only values this companion reports as measured are in Sections 5.4, 7 and 8.

```

{
  "ledger": "<instance name>",
  "input": {"compartments": 6, "fluxes": 11, "declared_conserved": 3},
  "predicates": {
    "Typed": {"status": "established", "witness": "declaration (Ty,ty,Cv)"},
    "Balanced": {"status": "established", "witness": "residual 2.2e-16, tol 1e-9"},
    "Conserved": {"status": "not established", "needs": "disturbance budget eps_1"},
    "Positive": {"status": "established", "witness": "donor limitation, 11 of 11
fluxes"},

```

```

"Admissible": {"status": "not applicable", "needs": "species table and declared
Omega"},
"Safe": {"status": "not established", "needs": "barriers and horizon"},
"Adequate service": {"status": "not established", "needs": "service definition"},
"Closed": {"status": "not established", "needs": "use vector D at tau_use"}
},
"programmes": {"closure": "not run (missing D)", "critical_margin": "not run
(missing c)"},
"concealed_deficit": {"z": 3.0, "delta_star": [4.0, 4.0], "solver": "highs"}},
"vintage": {"name": "RAM Legacy", "version": "4.66", "release_doi":
"10.5281/zenodo.14043031",
"issued": "2024-11-06", "licence": "CC-BY-4.0", "pull_archived": true,
"content_tests": {"spot_values_reproduced": "4 of 6", "internal_consistency": "35
of 35 to 1e-9"}}
}

```

Two keys are load-bearing and often omitted elsewhere. `witness` must name an object, not a feeling: a `established` without a witness is printed as `not established` by the tool. And `vintage.content_tests` must be a *result of running the checks of Section 7*, not a copy of the citation string, which is exactly the distinction the main text needed in its own data-availability statement.

7. Vintage: how to pin a number to a release, not to a citation

The main text quotes cohort statistics from an archived extract of the RAM Legacy Stock Assessment Database and pins them to release v4.66 (Zenodo record 14043031, issued 6 November 2024, CC-BY-4.0; the DOI is the record number in Zenodo’s standard form, and the issue date and licence are read from the record, not from the article). A reader cannot check that sentence by looking at the citation. They can check it by looking at the numbers, which is what the supplementary’s S5 does in three tests; all three are procedures worth generalising.

Test 1 — spot values. Take the release’s own published examples and recompute them from the download. On the archived cohort, four of six published fishing-mortality values reproduce v4.66 exactly to the printed precision: Adriatic anchovy 17–18 $F = 1.0026$ against a published 1.00; North Sea herring 0.2274 against 0.23; western Baltic herring (ICES 22–24) 0.193 against 0.19; Argentine anchovy south 0.0114 against 0.011. Two of the six example stocks no longer carry any series in public releases v4.64 to v4.66, so the test is reported as 4 of 6 with the reason for the other two, which is the honest form of the result: the release is confirmed and the release has moved since.

Test 2 — protocol recovery by grid. Where a statistic’s specification is not written down, search the small space of conventions instead of guessing. The grid here was over (zeros included or excluded) $\times$ (last finite year or last year) $\times$ (most-recent-assessment or all-year view) $\times$ (which F column), with the below-reference rule $B_lim = 0.2 \max SSB$. One cell reproduces both anchors of the record: on v4.66, 454 stocks, 69 zeros, median 3.3893 yr; on v4.44 (Zenodo 2542919, issued 22 December 2018 according to the record’s own metadata), 415, 63, 2.5683 yr. Two released medians pinned by one cell is a specification identified, not fitted.

Test 3 — internal consistency of the pull. Recompute the statistic on the archived extract itself. With B_lim confirmed row by row, all 35 positive rows of the archived 43-stock class cohort reproduce $ADH = F^{-1} \log(SSB/B_lim)$ to relative error below $1e-9$, and the eight zero rows are exactly the stocks with $SSB \leq B_lim$, with no zero row above the reference. This is the test that separates “the archived number is wrong” from “the archived number is about a different cohort”, and here it gives the second answer: the archived median is 1.7902 yr against 3.39 yr on the full v4.66 release, 33 of the 43 identifiers still exist in v4.66, the per-stock differences average 3.44 yr in absolute value, and the 33 stocks’ own archived-vintage median

of 2.79 yr becomes 2.34 yr on current series. A reader who does not run Test 3 will report a discrepancy where there is a definition.

Protocol 8 (what to archive). The pull, not the citation: the release identifier, the issue date, the licence, the file checksum, the date of download, and the code path that reads the file. The main text carries this as a data-availability statement; the tool carries it as the `vintage` key of Section 6. Either way the test of having done it is the same — a third party can be told *which* number to expect, and for how long.

Why this belongs in a methods companion rather than in an appendix: without it, every quantitative statement in this corpus is a claim about a moving object, and the standard remedy (cite the publisher) does not work, because the publisher is not the object that changed.

8. The reproduction bundle, its v2 relabelling, and the analysis record beside it

Three scripts, one manifest, one output record:

file	produces	reads	deterministic
<code>persistence_index_simulation.py</code>	the record-length table behind Proposition 32 (main text §6.5.1)	nothing — simulated inside the declared linear-trend class	yes, seed 7
<code>curvature_and_crossover.py</code>	the <code>kappa</code> table for Proposition 27, the closed-form check $T = H_{loc}/(1-kappa)$, the reserve-life crossover grid for Proposition 28, the three-law illustration of Proposition 26	nothing — the arithmetic is on figures tabulated in the article	yes, pure arithmetic
<code>certification_lp.py</code>	the compensation premium, the worst-concealed-deficit LP of Proposition 30, the event-time witness of Proposition 31, and the certificate vectors of the three classified indicators	nothing	yes

```
$ python3 revision/v7/code/certification_lp.py # and the other two, same directory
```

`code/MANIFEST.md` at v2 carries the versions, the run command, the relabelling table above and the scope limits, including one that matters for review: no script consumes the G3P basin rows whose provenance is recorded in the supplementary’s S5.4, because those rows are quarantined and nothing in the exhibits depends on them. The analysis record carries its own manifest in `source/MANIFEST.md`, and its scope limit is of the same kind: the two edition tables are not redistributed beyond the working copy needed to re-run the arithmetic, because the licence that permits the re-run is the same licence that requires the attribution the manifest records.

Environment as recorded at the head of `code/outputs.txt`: Python 3.13.14, numpy 2.3.5, scipy 1.17.1. The three scripts run in under ten seconds in total; there is no build step, no configuration file and no network call, which is what makes the outputs quotable.

What the bundle does not contain, and why that is a scope rule rather than a gap. The recomputation of the aggregate overshoot date — the arithmetic behind the main text’s Remark 37 and the supplementary’s S17, which reads two edition tables of the National Footprint and Biocapacity Accounts — lives beside the bundle as its own analysis record, `analysis/nfa_tau/`, with the script, three per-year CSVs, a revision ledger, the complete stdout, `checksums.txt`, the two tables as fetched under `source/` with their retrieval commands and hashes, and a `README.md` stating the licence and the release that could not be fetched. The reason is the manifest’s own rule quoted above: every script in the bundle reads nothing but

declared figures, and a program that ingests a downloaded table would break the property that makes the outputs quotable without a network. Separating them keeps each claim testable on its own terms: the three scripts run in seconds with no input at all, and the analysis record runs in about a second on the archived tables, reproducing its outputs byte for byte.

What v2 changed, and why nothing was overwritten. The bundle first shipped against an earlier draft, and its printed section labels named that draft’s numbering (Section 7.1 exhibit, (S8.1), Section 8.1 exhibit). The article has since moved those contents — the premium and the concealed-deficit programme are Section 10.1 with the supplementary’s S8, and the three classified indicators are Sections 6.5.1 to 6.5.3. Version 2 of the bundle restates the labels only:

v5 printed	v2 prints
Section 7.1 exhibit (S8.1), (S8.2), (S8.3)	Section 10.1 and supplementary S8 exhibit (main text 6.5.1), (6.5.2), (6.5.3)
Section 8.1 exhibit	Section 6.5.1 exhibit
python3 revision/v5/code/\{\ldots\}	python3 revision/v7/code/\{\ldots\}

The numerical statements in the scripts and their outputs are untouched, and the build record for v2 asserts that the values on re-run match the archived record line for line. The original v5 bundle is left exactly as it stands, so a reader holding the older deposit can still reproduce what it printed. Relabelling a printed label is a cosmetic change; quietly editing a deposited script is not, and the second is what a versioned directory exists to prevent.

The two tables the bundle prints are the ones quoted in the main text, and both are worth reading as procedures rather than as exhibits:

```
n index median index mean index q90 stock drop
100 0.000 0.211 0.812 98.9
1000 0.000 0.247 0.835 998.9
10000 0.000 0.206 0.953 9999.1
100000 0.000 0.246 0.999 99999.0
```

The index stays at the noise-to-trend scale while the stock implied by the same series falls by five orders of magnitude (the `n usable` column, omitted from the excerpt above, is 400 on every row): a statistic that is bounded on the declared class is not a horizon, and Proposition 32’s content is the bound, not the trend. The crossover grid makes the same point in the opposite direction for reserve-life ratios — proportional depletion gives $T \approx 276.310$ yr against a local horizon of 20.000 yr, constant depletion gives 1999.998 yr against 1999.998 yr, accelerating depletion $\phi(A) = cA^{0.4}$ gives 83.312 yr against 50.000 yr — so the sign of the frozen-rate error depends on the law, and a reporter who publishes one number per stock is publishing a law-free guess.

9. What the tool cannot buy

Four statements, each of which is a theorem in the main text and a design constraint here.

1. **Passing all eight predicates certifies the declaration, not the system.** The predicates are conditions on a typed ledger and its declared data; the article’s Proposition 42 states what no typing can buy — a conservation law does not cross a type class, so summing two certified ledgers is not a third certified ledger. A checker that could certify safety from declarations would refute Proposition 42. 2. **An aggregate can refute or alarm, never clear.** Proposition 30’s `delta*_j` is the largest compatible deficit, so any aggregate-only disclosure has an unbounded-but-computable concealment. The tool prints `delta*_j` beside every aggregate it is given, and refuses to print a componentwise verdict. 3. **Event times do not**

transport. Proposition 31’s two-start example (Section 5.4) is a refutation of the inference from an aggregate trajectory to a component crossing time. Any pipeline that computes a date from an aggregate and labels it a component’s date is wrong in a way no amount of precision fixes. 4. **A bounded index is not a horizon.** Proposition 32 bounds the anomaly-persistence index on the declared trend class; the exhibit table in Section 8 shows the bound coexisting with a five-order-of-magnitude change in the implied stock. Boundedness is about the statistic, and the statistic is about trend detection.

The practical reading of the four: the tool’s output is a *certificate about the paperwork*, and its most useful single line is often the **needs** field of a non-passing predicate — a list of what to ask for — rather than the status vector itself.

10. Extension points, and what each would require

Two obligations are not currently checkable by anyone, including us, and it is worth naming what would change that, so the gap is a specification rather than a silence.

Affinity (Admissible). The clause needs, per conversion, a species table listing the reactions a moiety actually undergoes and a declared set Ω of admissible flux directions; with those, the predicate is a sign-constrained LP per conversion and the tool would report a named conflict (the main text’s Proposition 40 states what an empty corridor comes with). Without them, the predicate is evaluated on energy and entropy sign constraints only, and the report says so. A public instance of the two objects does not exist in the domains surveyed here, which is why the extension is listed rather than implemented.

Closure (Closed) for published indicators. The programme needs a use vector D at a stated timescale and a process graph. Statistical products publish flows, not process graphs, so for public indicators **Closed** is not **established** and stays that way; the tool’s contribution is to make the missing object *nameable* — a publisher that ships a process graph can be asked for it by name, and one that does not can be reported as unable to support the predicate, which is a different sentence from “the indicator fails”.

A fourth programme. The fibre-maximisation the main text discusses for the multiplier set is not included here, because its exhibit is a proof-check rather than a decision procedure; a checker for it would need a declared tie-break over the polytope, which is a modelling choice, not a computation.

11. Deposition, and what to hand to an editor

The bundle and this companion are separate works from the article, and deposit them that way: a Zenodo *version* of an existing record is for the same work in a new state, while a *new record* is for a distinct work that will be cited on its own. Concretely, for the article line: a new version of the existing record for the main text at v39 with supplementary v10, the **code/** bundle at v2, and this companion; and a separate record for this companion if it is submitted as a methods paper. Each record should carry, as files: the compiled PDF, the LaTeX source, the markdown source of record, the supplementary, the bundle, and the run record. The licence of the underlying data does not transfer to the code and should not be implied: the RAM Legacy release this companion quotes is CC-BY-4.0, which permits the reproduction of the four spot values in Section 7 with attribution, and permits nothing about the archived extract itself, which is why the extract is described by its protocol rather than shipped.

Code availability, in the form the main text’s statement expects.

> The three exhibits of the certification apparatus — the persistence-index simulation, the curvature and > crossover arithmetic, and the linear-programme exhibit with

its certificate vectors — are implemented in `> persistence_index_simulation.py`, `curvature_and_crossover.py` and `certification_lp.py` in the deposited `> bundle code/` at version 2, with the complete stdout of a single run in `code/outputs.txt`. The scripts read `>` no data beyond the figures quoted in the article and contact no network; a fallback vertex-enumeration route is `>` used only when no LP solver is available, and the script prints which route produced each number.

Declarations

Funding. None declared.

Competing interest. The author declares no competing interest.

Software note. The bundled code is a reproduction exhibit, not a supported package: no installation, no API, no test suite beyond the printed assertions, and no claim of numerical robustness beyond the tolerances stated in the scripts. The v2 changes are labels, recorded in `revision/v7/code/make_bundle_v2.py`, which performs the substitution and re-runs all three scripts.

Use of AI assistance. Drafting and iterative review were assisted by GLM (Z.ai), Qwen (Alibaba Cloud) and DeepSeek AI, as in the main text.